ADOPT-03: Success Metrics

> **Series:** ADOPT — Observability Adoption & Maturity | **Notebook:** 3 of 6 | **Created:** March 2026 | **Last Updated:** 11/11/2026

Overview

What gets measured gets improved. This notebook defines the key success metrics for an observability practice: Mean Time to Detect (MTTD), Mean Time to Resolve (MTTR), change failure rate, problem count trends, and alert noise ratio. For each metric, we provide a baseline DQL query, explain how to interpret results, and describe how to track improvement over time. These metrics translate observability investment into language that leadership understands.

Table of Contents

1. Why Success Metrics Matter
2. Mean Time to Detect (MTTD)
3. Problem Duration (MTTR)
4. Problem Count Trends
5. Alert Quality
6. Deployment Frequency and Change Failure Rate
7. Establishing Baselines
8. Summary and Next Steps

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS (Grail). The DQL does not apply to Dynatrace Managed, which has no Grail.
Permissions	`storage:events:read`
Data	At least 7 days of detected problem data for meaningful baselines; deployment events for § 6
Audience	SREs, engineering managers, VP of Engineering, CTO

1. Why Success Metrics Matter

Observability platforms generate enormous volumes of data. Without defined success metrics, it is impossible to answer the fundamental question: **"Is our observability practice making us better?"**

Success metrics serve three purposes:

Purpose	Audience	Example
Operational improvement	SRE / Platform teams	MTTR reduced from 45 min to 12 min
Business justification	Leadership / Finance	60% fewer customer-impacting incidents
Continuous improvement	All teams	Alert noise ratio decreased from 65% to 15%

The DORA (DevOps Research and Assessment) metrics are the most widely used delivery benchmarks, and leadership will often ask for them. Use them carefully: DORA's stability metrics measure **deployments** — *change fail rate* is "the ratio of deployments that require immediate intervention following a deployment", and the metric once called MTTR "was renamed and redefined as failed deployment recovery time", which counts only recovery from a failed change. Dynatrace problem data covers every anomaly, whatever caused it. § 3 and § 6 say where the two line up and where they do not.

2. Mean Time to Detect (MTTD)

MTTD measures how long it takes from when a problem begins to when it is detected. In Dynatrace, Dynatrace Intelligence continuously analyzes telemetry and opens problems automatically. MTTD is the difference between problem start time and the timestamp when Dynatrace Intelligence created the problem event.

Why It Matters

- Lower MTTD means faster awareness of issues
- Automatic detection removes the wait for a user report or a person noticing a chart
- Tracking MTTD validates that your instrumentation and alerting are working

2.1 MTTD Over the Last 7 Days

Dynatrace Intelligence problems have an `event.start` timestamp — the beginning of the analysed anomaly window — and each problem emits a record per **status transition**. Detection lag is the gap between `event.start` and the timestamp of the **`CREATED`** transition specifically.

> **The trap this query exists to avoid.** Every transition record carries the same `event.start`, so `timestamp - event.start` means something different on each one. On a `CLOSED` record it is the problem's **total duration**; only on the `CREATED` record is it detection lag. Filtering by `event.status == "CLOSED"` and subtracting therefore reports duration while looking exactly like an MTTD query — it returns plausible numbers, just for the wrong quantity. Measured on a live tenant 08/25/2026, the two differ by roughly **35×** (152 min vs 4.3 min average over the same 7 days).
>
> This also dictates the data object. **`dt.davis.problems` keeps one record per problem — its latest state** — so a problem's `CREATED` record is there only when nothing has happened to the problem since (573 of 3,749 problems over 7 days on the validation tenant, 10/23/2026). MTTD computed on it would cover that subset only. Use `fetch events` with `event.kind == "DAVIS_PROBLEM"`, which keeps every transition.

```
// MTTD: gap between the start of the analysed window and problem CREATION.
// The CREATED transition is what makes this detection lag rather than
duration.
fetch events, from:-7d
| filter event.kind == "DAVIS_PROBLEM"
| filter event.status_transition == "CREATED"
| filter dt.davis.is_duplicate == false
| fieldsAdd detection_lag_minutes = (timestamp - event.start) / 1m
| summarize {
    avg_mttd_minutes = avg(detection_lag_minutes),
    median_mttd_minutes = median(detection_lag_minutes),
    p95_mttd_minutes = percentile(detection_lag_minutes, 95),
    problem_count = count()
  }
```

> **Interpreting MTTD results.** Dynatrace publishes no target for this number, so the bands below are community practice — calibrate them against your own baseline:
> - **Median under 5 minutes** — detection is keeping pace with the telemetry.
> - **Median 5-15 minutes** — usually fine; check whether slow detection clusters in one problem type.
> - **Median over 15 minutes** — review detector configuration and instrumentation coverage.
>
> Prefer the median to the average: a handful of slow detections pull the average up. On the validation tenant (10/01/2026, 4,586 problems over 7 days) the query returned a median of **3.2 min**, an average of **9.3 min**, and a p95 of **18.6 min**. If your result lands in the hundreds of minutes, suspect the query before the environment: that is the signature of measuring duration instead of detection lag.

3. Problem Duration (MTTR)

Problem duration — reported here as MTTR — is the time from a problem's start to its close, read from `resolved_problem_duration` on closed problems.

What It Measures, and What It Does Not

- **A problem closes when its events close.** In Dynatrace, "a problem is closed when all Davis events in the problem are closed, or when you close the problem manually."

Duration therefore measures how long the anomaly lasted — including issues that recovered on their own — not how long a team took to resolve an incident.

- **It is not DORA's recovery metric.** DORA's failed deployment recovery time covers only failures caused by a change. Problem duration covers every problem. Comparing the two against the same benchmark misleads both ways.
- **It is still the right trend line.** A falling median duration on the services you own is evidence that detection, routing and remediation are improving. Pair it with your incident tool's time-to-resolve if leadership needs a human-response number.

3.1 MTTR Trend Over 7 Days

```
// MTTR trend: average problem duration in hours, by the hour the problem
closed
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_duplicate == false
| filter maintenance.is_under_maintenance == false
| makeTimeseries avg_mttr_hours = avg(resolved_problem_duration / 1h),
time:event.end
```

3.2 MTTR Summary Statistics

```
// MTTR summary: average, median, and p95 resolution time in hours
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_duplicate == false
| filter maintenance.is_under_maintenance == false
| fieldsAdd duration_hours = resolved_problem_duration / 1h
| summarize {
    avg_mttr = avg(duration_hours),
    median_mttr = median(duration_hours),
    p95_mttr = percentile(duration_hours, 95),
    total_problems = count()
  }
```

3.3 MTTR by Problem Category

Different problem categories often have very different resolution times. Breaking MTTR down by category helps identify which problem types need process improvement.

```
// MTTR breakdown by problem category
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| filter dt.davis.is_duplicate == false
| fieldsAdd duration_hours = resolved_problem_duration / 1h
| summarize {
    avg_mttr = avg(duration_hours),
    median_mttr = median(duration_hours),
    problem_count = count()
  }, by:{event.category}
| sort median_mttr desc
```

> **Benchmarks.** Resist putting a DORA band next to this number. DORA's recovery benchmarks apply to failed deployment recovery time — recovery from a change that needed immediate intervention — and DORA publishes its benchmarks in its State of DevOps research. Set targets against your own 30-day baseline (§ 7), and if you need a DORA figure, compute it from deployment-correlated problems as in § 6.2.

4. Problem Count Trends

Tracking the total number of problems over time reveals whether your environment is becoming more stable or more volatile. A decreasing trend indicates that root causes are being addressed, not just symptoms.

4.1 Weekly Problem Trend

```
// Weekly problem trend over the last 30 days, bucketed by problem start
fetch dt.davis.problems, from:-30d
| filter dt.davis.is_duplicate == false
| summarize {problem_count = count()}, by:{week_start = bin(event.start,
168h)}
| sort week_start asc
```

4.2 Problems by Category Over Time

```
// Problem trend by category over the last 7 days, by problem start
fetch dt.davis.problems, from:-7d
| filter dt.davis.is_duplicate == false
| makeTimeseries problem_count = count(), time:event.start, interval:24h, by:
{event.category}
```

4.3 Top Recurring Problem Types

Identifying the most frequent problem types helps prioritize remediation efforts.

```
// Top 10 recurring problem types in the last 30 days
fetch dt.davis.problems, from:-30d
| filter dt.davis.is_duplicate == false
| summarize {occurrences = count()}, by:{event.name}
| sort occurrences desc
| limit 10
```

5. Alert Quality

Alert fatigue is one of the greatest threats to operational effectiveness. When teams are overwhelmed by noisy, duplicate, or non-actionable alerts, they stop responding — and real problems get missed.

Do not measure noise with `dt.davis.is_frequent_event`. Frequent issue detection is being phased out on the latest Dynatrace platform, which is improving "the individual alert sources so that frequent, false-positive alerts are not generated in the first place." On the validation tenant the flag was true on **0 of 15,227 problems** over 30 days, so a ratio built on it reports a healthy number regardless of how noisy alerting is. Measure what a responder experiences instead: problems that close before anyone could act, problems raised inside maintenance windows, and the same problem recurring on the same component.

5.1 Short-Lived and Maintenance-Window Problems

```
// Alert-quality signals over 7 days: short-lived problems and problems raised
during maintenance
fetch dt.davis.problems, from:-7d
| filter dt.davis.is_duplicate == false
| fieldsAdd duration_min = resolved_problem_duration / 1m
| summarize {
    problems = count(),
    closed_within_5min = countIf(duration_min &lt; 5),
    during_maintenance = countIf(maintenance.is_under_maintenance == true)
  }
| fieldsAdd short_lived_pct = round(toDouble(closed_within_5min) /
toDouble(problems) * 100, decimals: 1)
| fieldsAdd maintenance_pct = round(toDouble(during_maintenance) /
toDouble(problems) * 100, decimals: 1)
```

5.2 Which Problem Types Are Short-Lived

The aggregate tells you there is noise; the breakdown tells you where. A problem type that closes within five minutes almost every time is a tuning candidate, not an incident.

```
// Top problem types by volume, with the share that closed within 5 minutes,
30 days
fetch dt.davis.problems, from:-30d
| filter dt.davis.is_duplicate == false
| summarize {
    total = count(),
    closed_within_5min = countIf(resolved_problem_duration &lt; 5m)
  }, by:{event.name}
| fieldsAdd short_lived_pct = round(toDouble(closed_within_5min) /
toDouble(total) * 100, decimals: 1)
| sort total desc
| limit 10
```

5.3 The Same Problem on the Same Component

A problem that keeps recurring on one component is a known issue being re-reported, not new information.

```
// Recurring problems: the same problem type on the same component, 30 days.
// The component is the root-cause entity when Dynatrace identified one,
otherwise the first
// affected entity — many problems have no root cause, and grouping on the
root cause alone
// would lump every occurrence of a type together.
// root_cause.smartscape_entity and smartscape.affected_entities replace the
deprecated
// root_cause_entity_name and affected_entity_ids. Read the Smartscape fields
first; the classic
// fields still work and fill problems the Smartscape fields leave empty.
fetch dt.davis.problems, from:-30d
| filter dt.davis.is_duplicate == false
| fieldsAdd component = coalesce(root_cause.smartscape_entity[name],
root_cause_entity_name, smartscape.affected_entities[0][name],
affected_entity_ids[0])
| summarize {occurrences = count()}, by:{event.name, component}
| filter occurrences &gt;= 5
| sort occurrences desc
| limit 15
```

> **What to do with the results.** Dynatrace publishes no target ratio, so track these as trends: the short-lived share and the recurrence counts should fall as tuning lands. The levers, in the order to try them:
> - **Delay notification, not detection.** Problem-triggered workflows have a **Minimum duration** option, so a problem that resolves inside the window never notifies anyone (FAQ-21).
> - **Tune the detector** that raises the short-lived type — sensitivity, thresholds, or the anomaly detector itself (ALERT-02).
> - **Fix the recurring cause** for problems that repeat on one component; suppression only hides it.
> - **Use maintenance windows for planned work**, and treat a high maintenance share as a prompt to check whether detection is too eager outside those windows.

6. Deployment Frequency and Change Failure Rate

Deployment frequency and change failure rate (CFR) are DORA delivery metrics. DORA defines change fail rate as "the ratio of deployments that require immediate intervention following a deployment." Dynatrace cannot see your intervention decisions, but it can see

whether a problem opened on the same component shortly after a deployment — a reasonable, explicitly defined proxy.

Measuring CFR with Dynatrace

1. Count deployment events in a time window (this is also your deployment frequency)
2. For each deployment, check whether a problem started on one of its affected entities within a window after it (one hour below)
3. Divide deployments followed by a problem by all deployments

Both steps need deployment events. Send them from your pipeline through the Events API v2 with `eventType` `CUSTOM_DEPLOYMENT`, attached to the entities you deployed.

6.1 Deployment Event Count

```
// Count deployment events in the last 7 days
// CUSTOM_DEPLOYMENT is an event.type (the Events API v2 eventType), not an
event.kind.
// API-ingested events carry event.kind == "DAVIS_EVENT"; the kinds seen on
the events
// object are DAVIS_EVENT, DAVIS_PROBLEM, FLEET_EVENT and SYNTHETIC_EVENT, so
a filter on
// event.kind == "CUSTOM_DEPLOYMENT" matches nothing, whether or not you send
deployments.
fetch events, from:-7d
| filter event.type == "CUSTOM_DEPLOYMENT"
| summarize deployment_count = count()
```

6.2 Deployments Followed by a Problem

This query joins each deployment to problems on the same affected entity and counts the deployments that were followed by a problem within one hour.

```
// Change failure rate proxy: deployments followed within 1 hour by a problem
on the same entity
fetch events, from:-7d
| filter event.type == "CUSTOM_DEPLOYMENT"
| fields deploy_id = event.id, deploy_time = timestamp, entity =
smartscape.affected_entities[][id]
| expand entity
| join [
    fetch dt.davis.problems, from:-7d
    | filter dt.davis.is_duplicate == false
    | fields problem_id = event.id, problem_start = event.start, entity =
smartscape.affected_entities[][id]
    | expand entity
  ], kind:leftOuter, on:{entity}, fields:{problem_id, problem_start}
| fieldsAdd followed = isNotNull(problem_start) and problem_start &gt;=
deploy_time and problem_start &lt;= deploy_time + 1h
| summarize {followed = max(followed)}, by:{deploy_id}
| summarize {deployments = count(), followed_by_problem = countIf(followed)}
| fieldsAdd change_failure_rate_pct = round(toDouble(followed_by_problem) /
toDouble(deployments) * 100, decimals: 1)

// Validated 10/01/2026. The validation tenant had no CUSTOM_DEPLOYMENT
events, so the join was
// exercised with event.type == "CUSTOM_INFO" in their place (11,606 events,
24 h) to prove the
// mechanics; with no deployment events it returns deployments = 0. The window
(1h) and the entity match
// are choices — widen the window for slow-burning failures, and expect a
problem that merely
// coincides with a deployment to count as a failure.
//
// Both sides join on Smartscape node IDs (smartscape.affected_entities
replaces the deprecated
// affected_entity_ids). The classic field still works but names ENVIRONMENT-
0000000000000001 for
// entities that exist only in Smartscape and for environment-level events, so
a classic join
// matches unrelated events through the environment: re-run 11/11/2026 with
CUSTOM_INFO over 24 h,
// the classic form matched 8 of 1,786 events to a problem, 7 of them only
through the environment;
// this form matched 1. Keep both sides on the same field — the ID spaces
differ.
```

> **Benchmarks.** DORA publishes change-fail-rate benchmarks in its State of DevOps research — take the current figures from dora.dev rather than from a table copied into a notebook. The proxy above counts any problem on a deployed entity within the window, so it will read higher than a rate based on deliberate rollback or hotfix decisions.

7. Establishing Baselines

A baseline is a snapshot of your current metrics that serves as the starting point for improvement tracking. Without baselines, you cannot measure progress.

Baseline Template

Record your current values and set improvement targets:

Metric	Current Baseline	3-Month Target	6-Month Target
MTTD	___ minutes	___ minutes	___ minutes
MTTR	___ hours	___ hours	___ hours
Weekly Problem Count	___ problems	___ problems	___ problems
Short-Lived Problem Share	___%	___%	___%
Change Failure Rate	___%	___%	___%

Best Practices for Baseline Setting

- Use a **30-day window** for the initial baseline to smooth out anomalies
- Exclude maintenance windows and known outage periods
- Filter out duplicate problems (`dt.davis.is_duplicate`) for cleaner metrics
- Record baselines per team or service if organizational structure allows
- **Re-baseline quarterly** to account for growth and environmental changes

8. Summary and Next Steps

Key Takeaways

- MTTD and problem duration (MTTR) are the two operational metrics to track first — and problem duration is not DORA's recovery metric
- Measure alert quality by short-lived and recurring problems, not by the frequent-issue flag
- Problem count trends reveal whether your environment is stabilizing or degrading
- Deployment frequency and a deployment-correlated CFR proxy bridge observability and DORA, provided deployments are sent as events
- Baselines must be established before improvement can be measured

Next Steps

- Proceed to **ADOPT-04: Team Enablement** to build role-based learning paths for your organization
- Create a recurring notebook or dashboard that runs these queries weekly
- Share baseline metrics with leadership to establish accountability

References

- Root cause analysis concepts (DT docs) — *"A problem is closed when all Davis events in the problem are closed, or when you close the problem manually."*
- Transition from frequent issue detection (DT docs) — *"Frequent issue detection is being phased out on the latest Dynatrace platform."*
- DORA's software delivery performance metrics (dora.dev) — *"The ratio of deployments that require immediate intervention following a deployment."*
- A history of DORA's software delivery metrics (dora.dev) — *"was renamed and redefined as failed deployment recovery time"*

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace

documentation.